Slides 17/12

Méthodes efficaces pour l’estimation de modèles spatio-temporel

Alexandre Loret

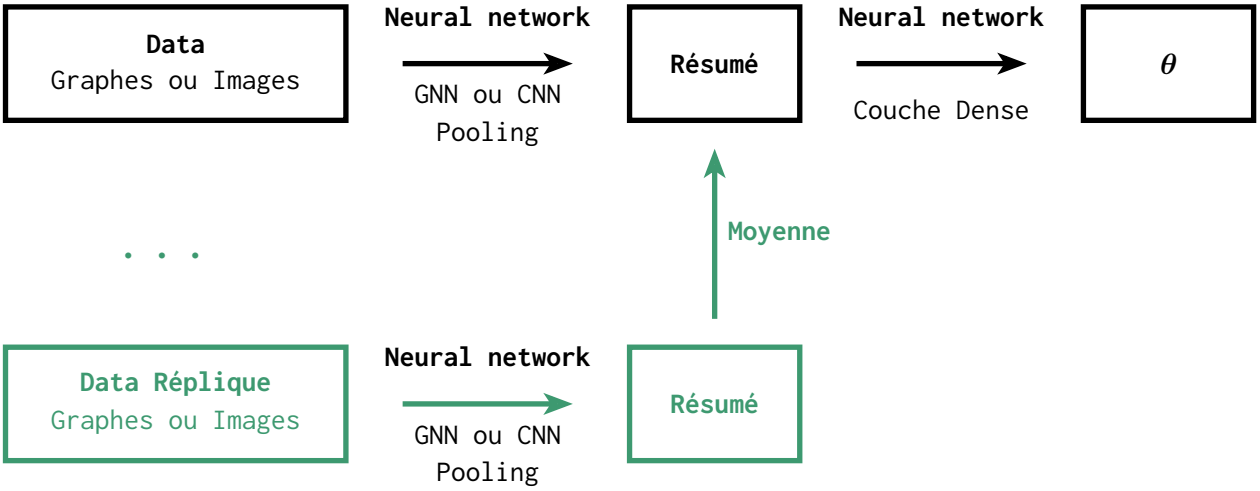
Table des matières

1 Détails méthodes 3
1.a Deepset 4
1.b Neural Likelihood Suface (NLS) 5
1.c Baseline 6
2 Petites corrections dans le cas Spatial 7
2.a Optim NLS sigmoïd 8
2.b Crash GNN Sainsbury-Dale 10
3 Spatio-Temporel 11
3.a Génération des données 2D 12
3.b Problème : Temps 15
3.c Entraînement rapide 16

1 Détails méthodes

Deepset

Les données répliquées peuvent (générées sur un même θ) peuvent mener à du biais.

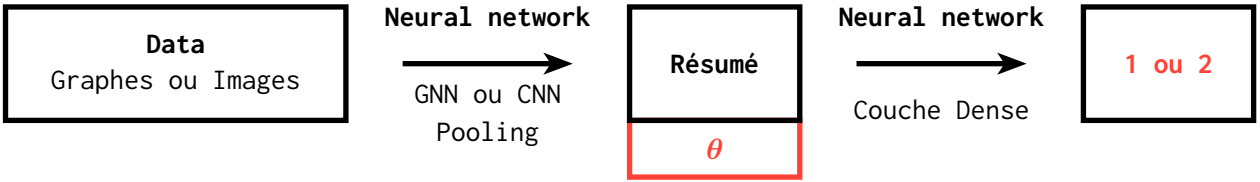


Neural Likelihood Suface (NLS)

Il a été montré que la vraisemblance peut être approchée de la façon suivante :

$$L(\theta|y) = \frac{h(y,\theta)}{1-h(y,\theta)}$$

avec $h(y,\theta)$ un classifier qui associe $C=1$ si la réalisation y et le paramètre θ sont dépendant, $C=2$ si indépendants.



Ce classifier permet de trouver la vraisemblance pour un couple (donnée, θ). Pour une donnée, le meilleur θ est trouvé par gridsearch ou optimisation.

Baseline

1. θ_{train} la moyenne de tout les θ utilisés dans le train.
2. On regarde le MAE de chaque θ du train avec ce θ_{train}
3. La baseline est la moyenne de ces MAE

2 Petites corrections dans le cas Spatial

Optim NLS sigmoïd

Par grid-search :

```
Time to test : 202.58s
Test loss : [0.04818347 0.11207985] ± [0.03773961 0.09932275]
```

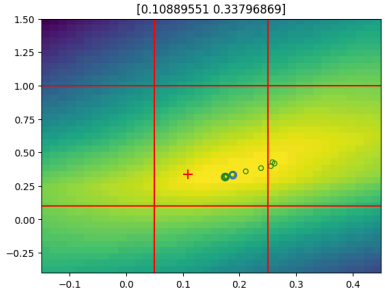
Utilisation du package `Pytorch-minimize` avec la méthode `bfgs`

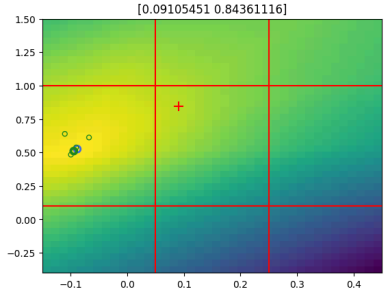
```
      Time to test : 114.88s
Avant Test loss : [0.06215605 0.2037503 ] ± [0.04539046 0.13381939]

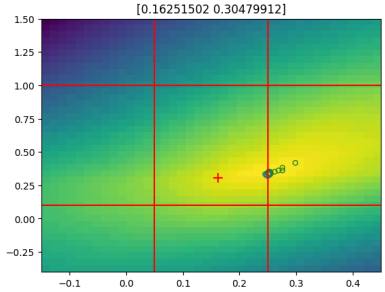
           Time to test : 157.98s
Maintenant Test loss : [0.0662095  0.14215676] ± [0.0500213  0.13185776]

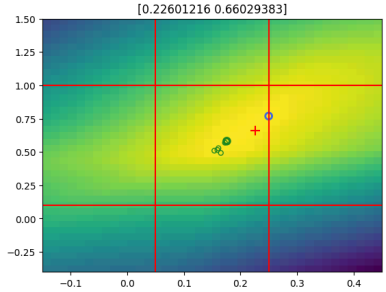
             Time to test : 195.93s
Avec Sigmoid Test loss : [0.05142049 0.11565374] ± [0.03823776 0.09935459]
```

Optim NLS sigmoïd (ii)









Crash GNN Sainsbury-Dale

Retester avec la dernière modification (seule la loss moyenne est enregistrée)

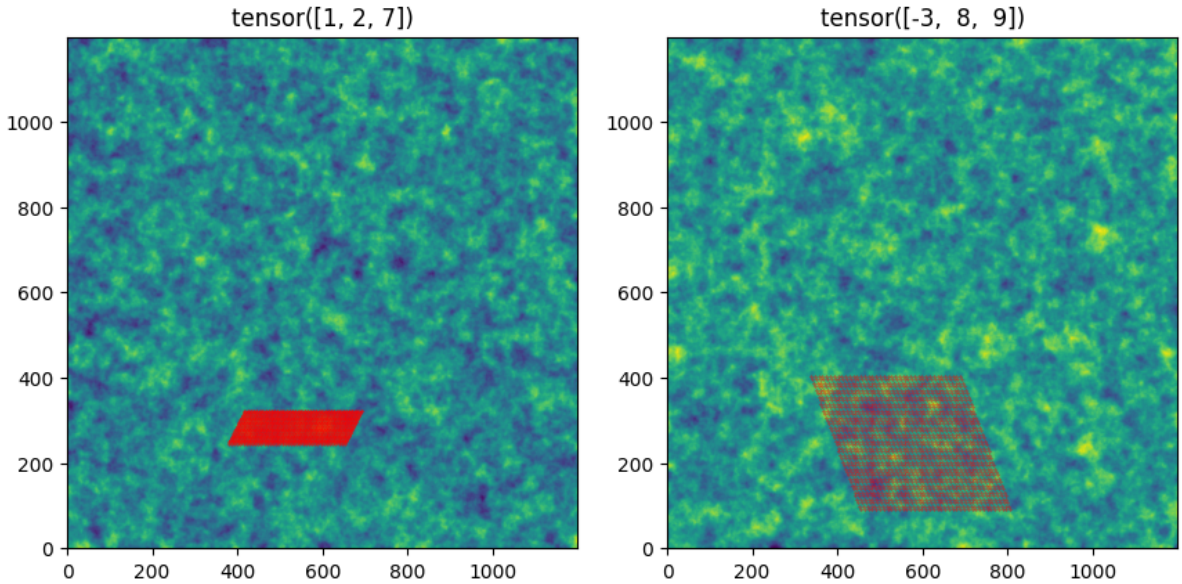
3 Spatio-Temporel

Génération des données 2D

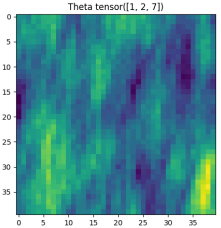
2 méthodes :

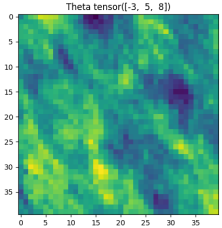
1. Méthodes mentionnée la semaine dernière
   1. Génération (et sauvegarde) rapide de 2000 images 1200×1200 avec des ρ_s et ρ_t similaires pour chaque images
   2. Lors de la création du dataset :
      - Récupère n le nombre θ, m la taille du deepset, et la taille des images de sortie
      - Génère n θ (advection $\{-3, ...3\}$, range temporel $\{2, ...10\}$, range spatial $\{2, ...10\}$)
   3. Pour récupérer les données :
      - prend en entrée un `idx` dans $\{1, ...n\}$
      - récupère le θ associé
      - récupère m images 1200×1200
      - Pour chaque image, extrait une à une position aléatoire avec les paramètres θ
2. Méthode par génération sur le tas
   1. Similaire à l'étape 2
   2. Génère une image de bonne taille directement (prend en compte l'advection)

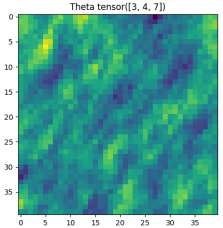
Génération des données 2D (ii)

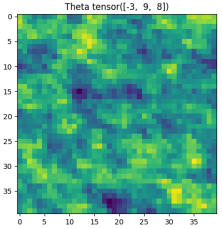


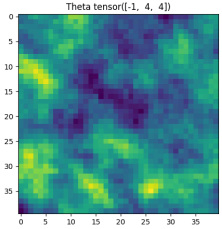
Génération des données 2D (iii)

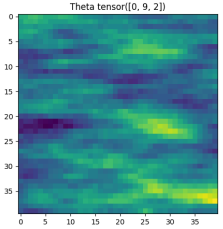












Problème : Temps

`batch_size` = 5, `deepset_size` = 4, `theta_size` = 200

Méthode mentionnée la semaine dernière : `33.488s`

Méthode différente avec `PrecisionOpMatrix` : `6.343s`

Méthode différente avec `PrecisionOp` : `90.681s`

Entraînement rapide

